MANAV RACHNA INTERNATIONAL INSTITUTE OF RESEARCH & STUDIES
School of Computer Applications (SCA)

LAB PRACTICAL RECORD FILE

DATA STRUCTURES & ALGORITHMS

SUBMITTED TO:
Mr. Sachin Sharma
Faculty / Assistant Professor
School of Computer Applications
MRIIRS

SUBMITTED BY:
Vanshika
Roll No: 26MCAP101N046
Course: MCA
Semester: 1

Academic Session 2026 - 2027

INDEX OF EXPERIMENTS

Exp. No.	Date	Title of Experiment	Page No.	Sign & Remarks
Exp 1	14-10-2026	List Operations (Insertion, Deletion and Display)	4-7	
Exp 2A	14-10-2026	Quick Sort Algorithm	8-9	
Exp 2B	14-10-2026	Merge Sort Algorithm	10-11	
Analysis	14-10-2026	Time Complexity Comparison (Quick Sort vs Merge Sort)	12-13	
Exp 3A	14-10-2026	Stack using List	14-16	
Exp 3B	14-10-2026	Queue using List	17-19	
Exp 3C	14-10-2026	Circular Queue using List	20-22	
Exp 3D	14-10-2026	Browser History using Stack	23-26	
Exp 4	14-10-2026	Evaluate Postfix Expression	27-28	
Exp 5	14-10-2026	Convert Infix to Postfix	29-30	
Exp 6	14-10-2026	Singly Linked List (Insert, Delete, Display)	31-35	
Exp 7	14-10-2026	Circular Linked List (Insert, Delete, Display)	36-40	
Exp 8A	14-10-2026	Stack using Linked List	41-44	
Exp 8B	14-10-2026	Queue using Linked List	45-49	
Exp 8C	14-10-2026	Circular Queue using Linked List	50-54	
Exp 9A	14-10-2026	Concatenation of Two Linked Lists	55-57	
Exp 9B	14-10-2026	Polynomial Addition using Linked Lists	58-60	
Exp 10	14-10-2026	Doubly Linked List (Insert, Delete, Display)	61-65	

EXPERIMENT 1: LIST OPERATIONS (INSERTION, DELETION AND DISPLAY)

AIM: To write and execute a Python program to perform basic operations on a list: inserting an element at a specified position, deleting an element by position, and displaying the list contents.

Python Source Code

```
# Experiment 1: List Operations (Insertion, Deletion and Display)
a = []

n = int(input("How many elements? "))
for i in range(n):
    x = int(input(f"Enter element {i + 1}: "))
    a.append(x)

while True:
    print("\n--- MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        pos = int(input("Enter position (1-based): "))
        value = int(input("Enter value: "))
        if 1 <= pos <= len(a) + 1:
            a.insert(pos - 1, value)
            print(f"Inserted {value} at position {pos}.")
        else:
            print("Invalid position!")

    elif choice == 2:
        pos = int(input("Enter position to delete (1-based): "))
        if 1 <= pos <= len(a):
            removed = a.pop(pos - 1)
            print(f"Deleted {removed} from position {pos}.")
        else:
            print("Invalid position!")

    elif choice == 3:
        print("Current List:", a)

    elif choice == 4:
        print("Exiting...")
        break
    else:
        print("Invalid choice! Please select 1-4.")
```

Space for Output

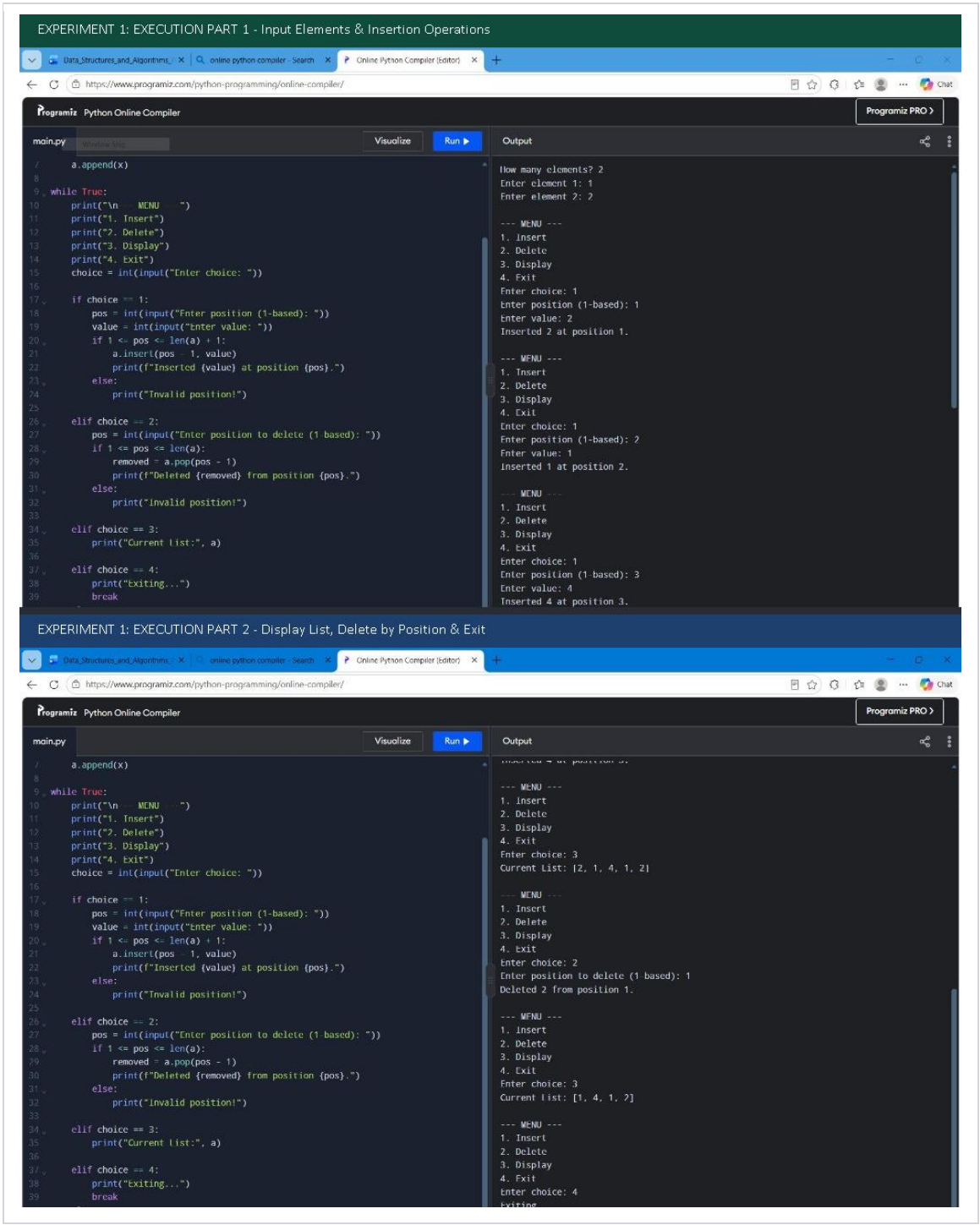


Figure: Execution output of List Operations (Insertion, Deletion and Display)

Sample Run: Execution Log / Console Output (Terminal Execution):

```
How many elements? 2
Enter element 1: 1
Enter element 2: 2
```

```
--- MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter position (1-based): 1
Enter value: 2
Inserted 2 at position 1.

--- MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter position (1-based): 2
Enter value: 1
Inserted 1 at position 2.

--- MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter position (1-based): 3
Enter value: 4
Inserted 4 at position 3.

--- MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
Current List: [2, 1, 4, 1, 2]

--- MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 2
Enter position to delete (1-based): 1
Deleted 2 from position 1.

--- MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
Current list: [1, 4, 1, 2]

--- MENU ---
```

```
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 4
Exiting...
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 2A: QUICK SORT ALGORITHM

AIM: To write and execute a Python program to sort a list of numbers using the Quick Sort divide-and-conquer algorithm.

Python Source Code

```
# Experiment 2A: Quick Sort Algorithm
def quick_sort(a):
    if len(a) <= 1:
        return a

    pivot = a[-1]
    left = []
    right = []

    for x in a[:-1]:
        if x <= pivot:
            left.append(x)
        else:
            right.append(x)

    return quick_sort(left) + [pivot] + quick_sort(right)

# Driver Code
a = list(map(int, input("Enter numbers separated by spaces: ").split()))
print("Original List:", a)
sorted_a = quick_sort(a)
print("Sorted list (Quick Sort):", sorted_a)
```

Space for Output

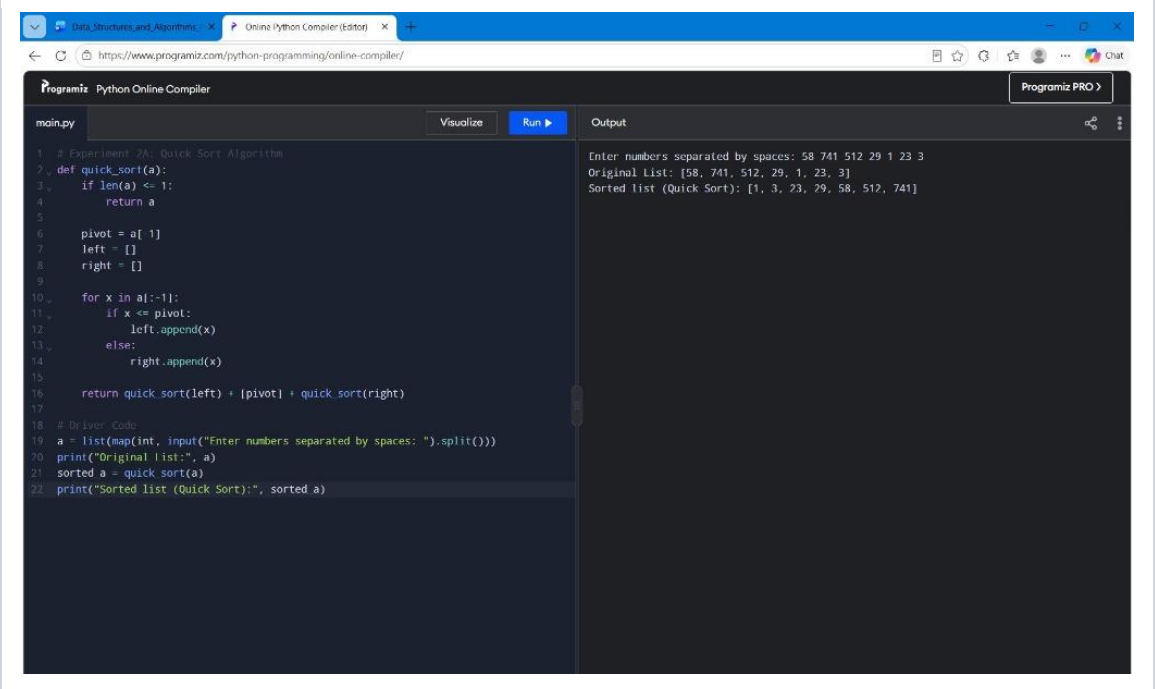


Figure: Execution output of Quick Sort Algorithm

Sample Run: Execution Log / Console Output (Terminal Execution):

```
Enter numbers separated by spaces: 58 741 512 29 1 23 3
Original List: [58, 741, 512, 29, 1, 23, 3]
Sorted list (Quick Sort): [1, 3, 23, 29, 58, 512, 741]
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 2B: MERGE SORT ALGORITHM

AIM: To write and execute a Python program to sort a list of numbers using the Merge Sort divide-and-conquer algorithm.

Python Source Code

```
# Experiment 2B: Merge Sort Algorithm
def merge_sort(a):
    if len(a) <= 1:
        return a

    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    result = []

    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Driver Code
a = list(map(int, input("Enter numbers separated by spaces: ").split()))
print("Original List:", a)
sorted_a = merge_sort(a)
print("Sorted list (Merge Sort):", sorted_a)
```

Space for Output

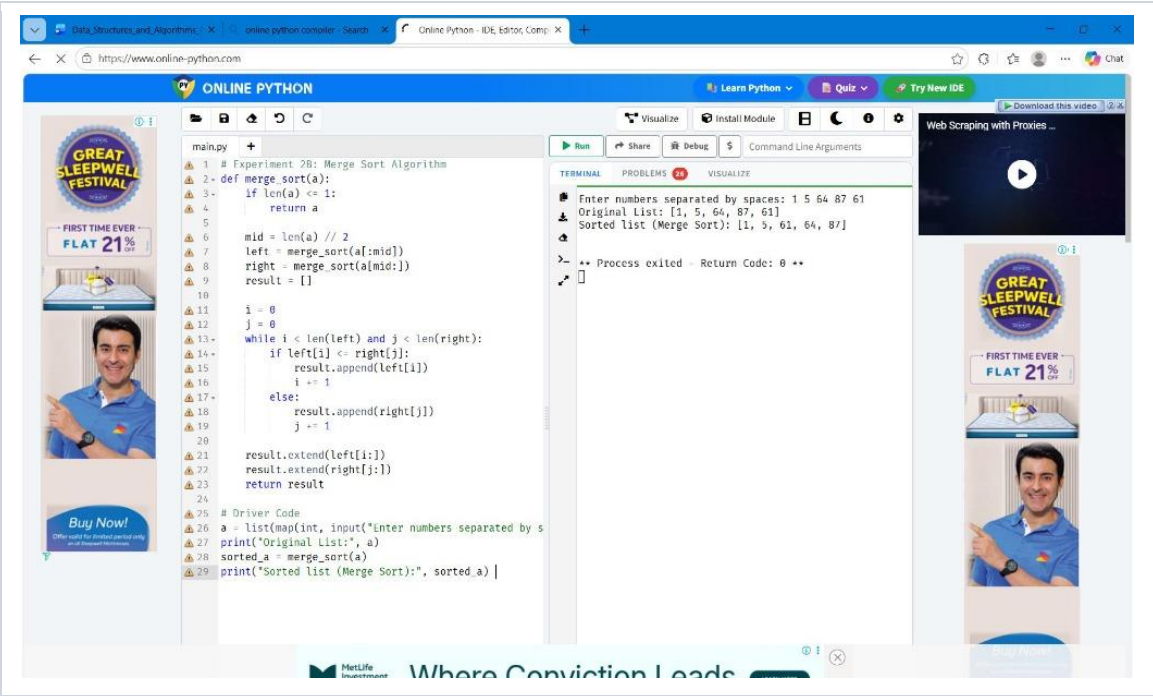


Figure: Execution output of Merge Sort Algorithm

Sample Run: Execution Log / Console Output (Terminal Execution):

```
Enter numbers separated by spaces: 1 5 64 87 61
Original List: [1, 5, 64, 87, 61]
Sorted list (Merge Sort): [1, 5, 61, 64, 87]
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

TIME COMPLEXITY COMPARISON: TIME COMPLEXITY COMPARISON: QUICK SORT VS. MERGE SORT

AIM: To analytically compare the time complexity, space complexity, stability, and operational efficiency of Quick Sort and Merge Sort.

Algorithmic Comparison Matrix

Algorithm	Best Time	Avg Time	Worst Time	Aux Space	Stability	In-Place
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	No

Python Source Code

```
# Analytical Demonstration: Quick Sort vs Merge Sort Performance Characteristics
# Quick Sort: In-place, cache-friendly, but O(n^2) worst case if pivot is unbalanced.
# Merge Sort: Guaranteed O(n log n) in all cases, stable, but requires O(n) auxiliary space.

import time
import random

def quick_sort(a):
    if len(a) <= 1:
        return a
    pivot = a[-1]
    left = [x for x in a[:-1] if x <= pivot]
    right = [x for x in a[:-1] if x > pivot]
    return quick_sort(left) + [pivot] + quick_sort(right)

def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    res, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            res.append(left[i]); i += 1
        else:
            res.append(right[j]); j += 1
    res.extend(left[i:]); res.extend(right[j:])
    return res

test_data = [random.randint(1, 1000) for _ in range(2000)]
t0 = time.perf_counter()
quick_sort(test_data.copy())
t_quick = time.perf_counter() - t0

t0 = time.perf_counter()
```

```
merge_sort(test_data.copy())
t_merge = time.perf_counter() - t0

print(f"Quick Sort Execution Time for 2000 items: {t_quick * 1000:.3f} ms")
print(f"Merge Sort Execution Time for 2000 items: {t_merge * 1000:.3f} ms")
```

Space for Output

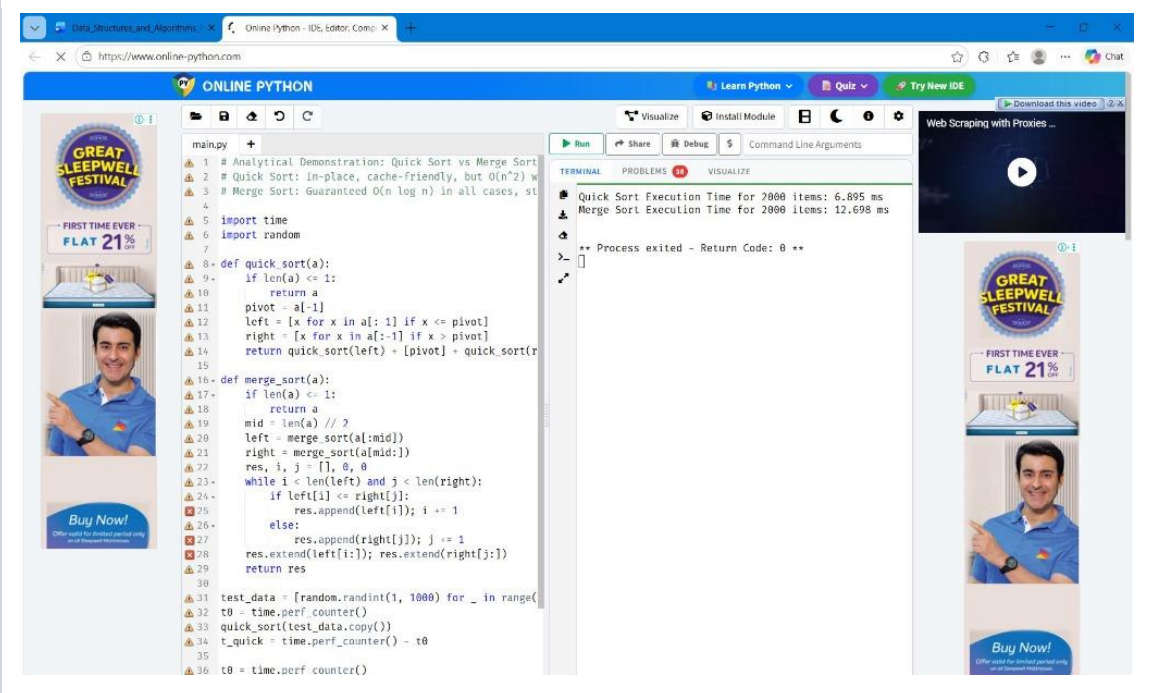


Figure: Execution output of Time Complexity Comparison (Quick Sort vs Merge Sort)

Sample Run: Execution Log / Console Output (Terminal Execution):

```
Quick Sort Execution Time for 2000 items: 6.895 ms
Merge Sort Execution Time for 2000 items: 12.698 ms

** Process exited - Return Code: 0 **
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 3A: STACK IMPLEMENTATION USING LIST

AIM: To implement the Stack data structure (Last In First Out - LIFO) using a Python list supporting Push, Pop, and Display operations.

Python Source Code

```
# Experiment 3A: Stack using List
stack = []

while True:
    print("\n--- STACK MENU ---")
    print("1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        x = int(input("Enter value: "))
        stack.append(x)
        print(f"{x} pushed onto stack.")
    elif choice == 2:
        if len(stack) == 0:
            print("Stack is empty (Underflow)!")
        else:
            print("Popped element:", stack.pop())
    elif choice == 3:
        if len(stack) == 0:
            print("Stack is empty.")
        else:
            print("Stack (top to bottom):", stack[::-1])
    elif choice == 4:
        print("Exiting...")
        break
    else:
        print("Invalid choice!")
```

Space for Output

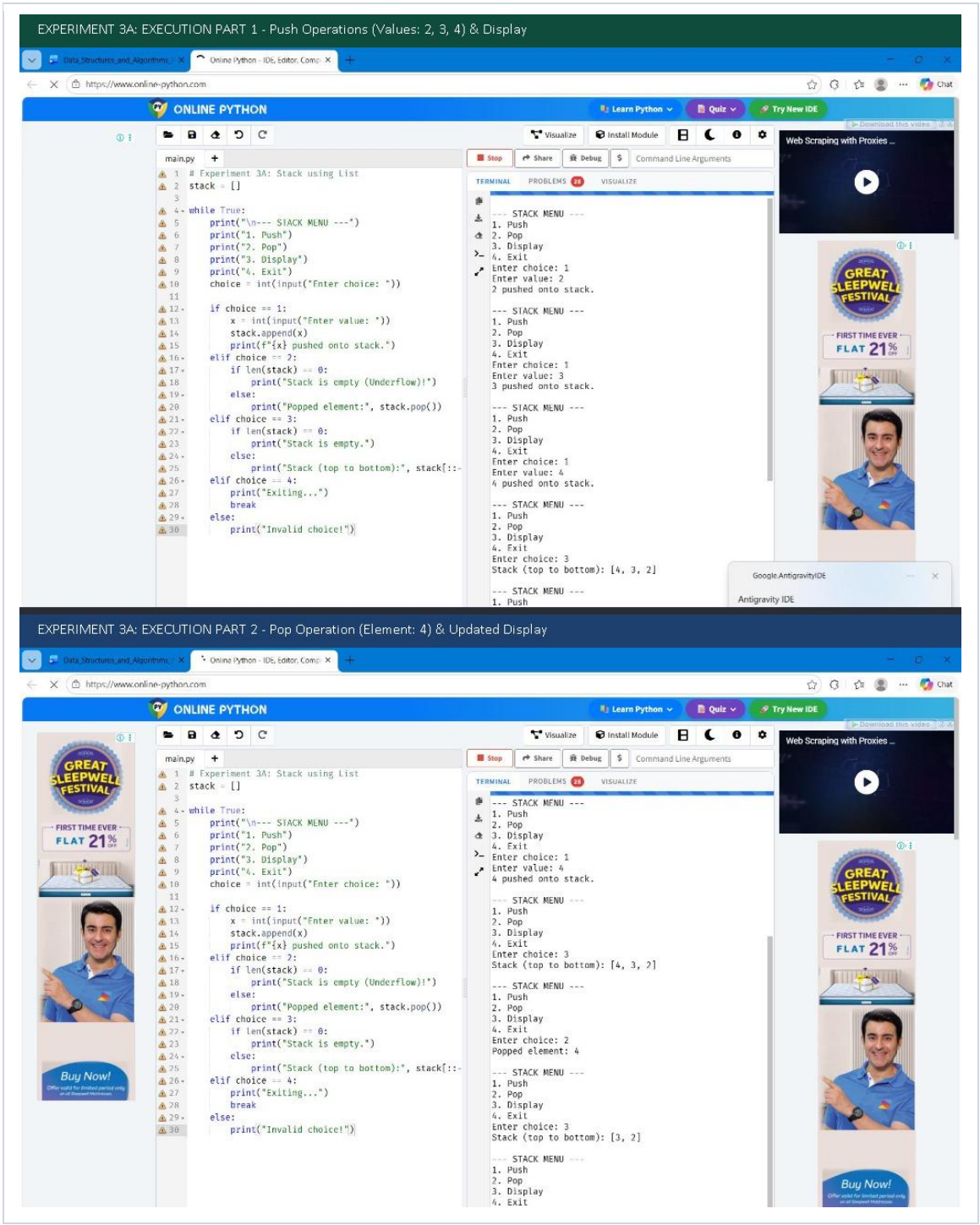


Figure: Execution output of Stack using List

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- STACK MENU ---
1. Push
2. Pop
```

```
3. Display
4. Exit
Enter choice: 1
Enter value: 2
2 pushed onto stack.

--- STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter value: 3
3 pushed onto stack.

--- STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter value: 4
4 pushed onto stack.

--- STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack (top to bottom): [4, 3, 2]

--- STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
Popped element: 4

--- STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack (top to bottom): [3, 2]
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 3B: QUEUE IMPLEMENTATION USING LIST

AIM: To implement the Linear Queue data structure (First In First Out - FIFO) using a Python list supporting Enqueue, Dequeue, and Display operations.

Python Source Code

```
# Experiment 3B: Queue using List
queue = []

while True:
    print("\n--- QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        x = int(input("Enter value: "))
        queue.append(x)
        print(f"{x} enqueued successfully.")
    elif choice == 2:
        if len(queue) == 0:
            print("Queue is empty (Underflow)!")
        else:
            print("Deleted element:", queue.pop(0))
    elif choice == 3:
        if len(queue) == 0:
            print("Queue is empty.")
        else:
            print("Queue (front to rear):", queue)
    elif choice == 4:
        print("Exiting...")
        break
    else:
        print("Invalid choice!")
```

Space for Output

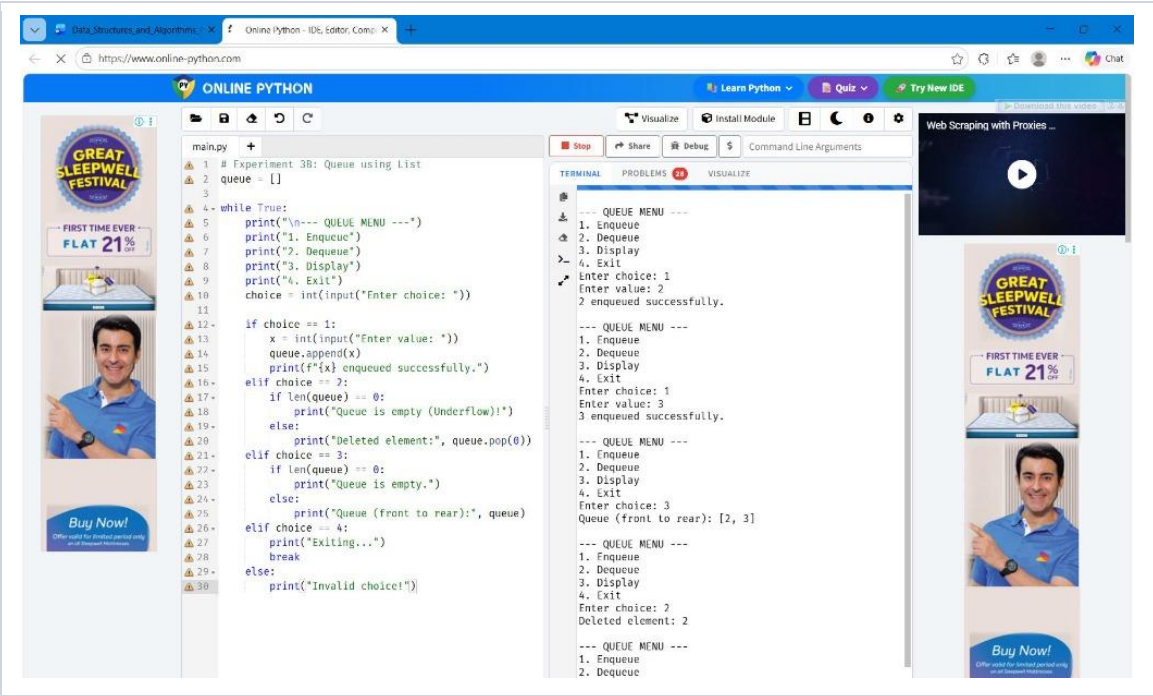


Figure: Execution output of Queue using List

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter value: 2
2 enqueued successfully.

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter value: 3
3 enqueued successfully.

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 3
Queue (front to rear): [2, 3]

--- QUEUE MENU ---
1. Enqueue
```

```
2. Dequeue
3. Display
4. Exit
Enter choice: 2
Deleted element: 2
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 3C: CIRCULAR QUEUE IMPLEMENTATION USING LIST

AIM: To implement a fixed-size Circular Queue using a Python list with modular arithmetic to prevent memory wastage.

Python Source Code

```
# Experiment 3C: Circular Queue using List
size = 5
queue = [None] * size
front = -1
rear = -1

while True:
    print("\n--- CIRCULAR QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        if (rear + 1) % size == front:
            print("Queue is full (Overflow)!")
        else:
            x = int(input("Enter value: "))
            if front == -1:
                front = 0
            rear = (rear + 1) % size
            queue[rear] = x
            print(f"Inserted {x} at index {rear}.")

    elif choice == 2:
        if front == -1:
            print("Queue is empty (Underflow)!")
        else:
            print("Deleted element:", queue[front])
            queue[front] = None
            if front == rear:
                front = rear = -1
            else:
                front = (front + 1) % size

    elif choice == 3:
        if front == -1:
            print("Queue is empty.")
        else:
            print("Circular Queue elements:", end=" ")
            i = front
            while True:
                print(queue[i], end=" ")
                if i == rear:
                    break
                i = (i + 1) % size
            print()
```

```
    elif choice == 4:
        print("Exiting...")
        break
    else:
        print("Invalid choice!")
```

Space for Output

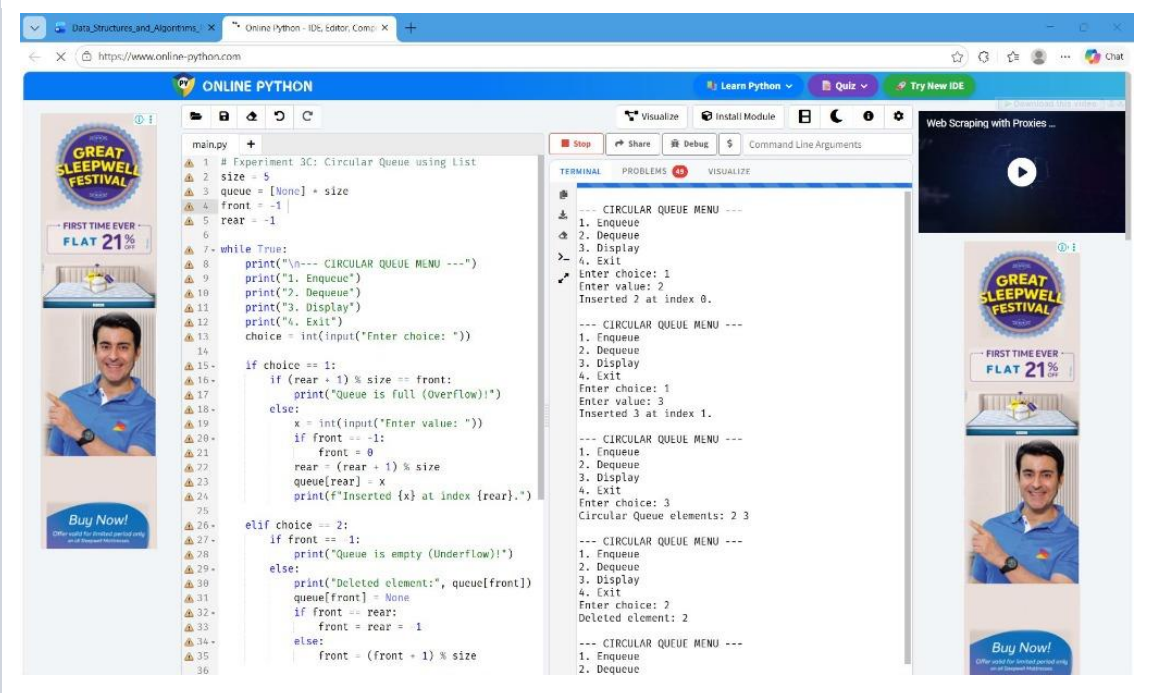


Figure: Execution output of Circular Queue using List

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter value: 2
Inserted 2 at index 0.

--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter value: 3
Inserted 3 at index 1.

--- CIRCULAR QUEUE MENU ---
```

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 3
Circular Queue elements: 2 3

--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 2
Deleted element: 2
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 3D: SIMPLE BROWSER HISTORY NAVIGATION USING STACK

AIM: To simulate web browser navigation history (visiting pages, going back, and viewing current URL) using the Stack concept.

Python Source Code

```
# Experiment 3D: Simple Browser History using Stack
history = []
current = -1

while True:
    print("\n--- BROWSER NAVIGATION MENU ---")
    print("1. Visit page")
    print("2. Back")
    print("3. Current page")
    print("4. Full History")
    print("5. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        page = input("Enter page URL: ")
        history = history[:current + 1]
        history.append(page)
        current += 1
        print(f"Visited: {page}")

    elif choice == 2:
        if current > 0:
            current -= 1
            print(f"Moved BACK to: {history[current]}")
        else:
            print("No previous page in history!")

    elif choice == 3:
        if current == -1:
            print("No page currently opened.")
        else:
            print("Current page:", history[current])

    elif choice == 4:
        print("History Stack:", history, f"| Active Index: {current}")

    elif choice == 5:
        print("Exiting browser simulation...")
        break
    else:
        print("Invalid choice!")
```

Space for Output

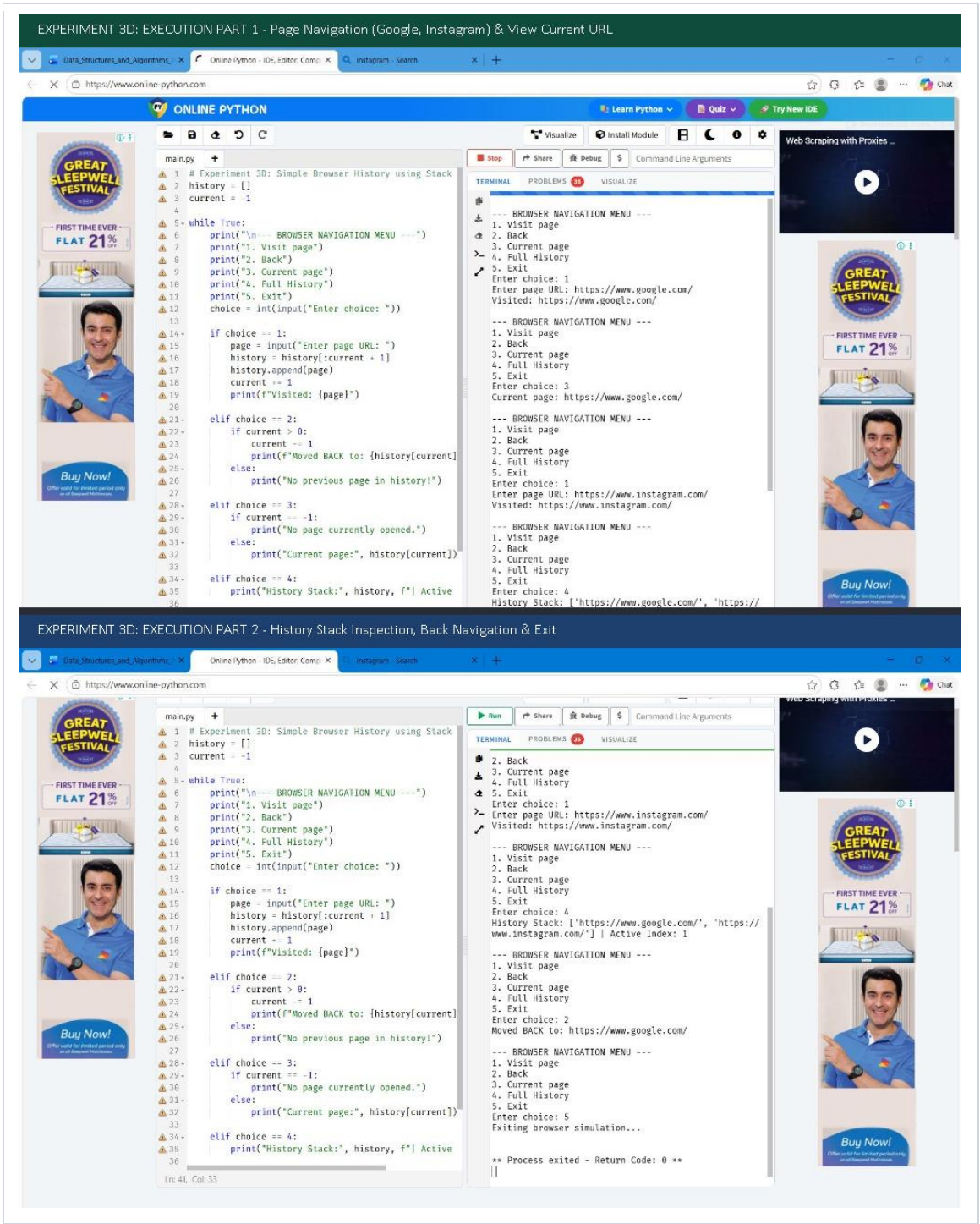


Figure: Execution output of Browser History using Stack

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- BROWSER NAVIGATION MENU ---
1. Visit page
2. Back
```

```
3. Current page
4. Full History
5. Exit
Enter choice: 1
Enter page URL: https://www.google.com/
Visited: https://www.google.com/

--- BROWSER NAVIGATION MENU ---
1. Visit page
2. Back
3. Current page
4. Full History
5. Exit
Enter choice: 3
Current page: https://www.google.com/

--- BROWSER NAVIGATION MENU ---
1. Visit page
2. Back
3. Current page
4. Full History
5. Exit
Enter choice: 1
Enter page URL: https://www.instagram.com/
Visited: https://www.instagram.com/

--- BROWSER NAVIGATION MENU ---
1. Visit page
2. Back
3. Current page
4. Full History
5. Exit
Enter choice: 4
History Stack: ['https://www.google.com/', 'https://www.instagram.com/'] | Active Index: 1

--- BROWSER NAVIGATION MENU ---
1. Visit page
2. Back
3. Current page
4. Full History
5. Exit
Enter choice: 2
Moved BACK to: https://www.google.com/

--- BROWSER NAVIGATION MENU ---
1. Visit page
2. Back
3. Current page
4. Full History
5. Exit
Enter choice: 5
Exiting browser simulation...

** Process exited - Return Code: 0 **
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 4: EVALUATE POSTFIX EXPRESSION USING STACK

AIM: To evaluate a Postfix (Reverse Polish) expression using a stack in Python.

Python Source Code

```
# Experiment 4: Evaluate Postfix Expression
exp = input("Enter postfix expression (e.g. 23*54*+): ")
stack = []

for ch in exp:
    if ch.isdigit():
        stack.append(int(ch))
    elif ch in "+-*/^":
        if len(stack) < 2:
            print("Invalid expression!")
            break
        b = stack.pop()
        a = stack.pop()

        if ch == '+':
            stack.append(a + b)
        elif ch == '-':
            stack.append(a - b)
        elif ch == '*':
            stack.append(a * b)
        elif ch == '/':
            if b == 0:
                print("Cannot divide by zero!")
                break
            stack.append(a // b)
        elif ch == '^':
            stack.append(a ** b)
    elif ch == ' ':
        continue
    else:
        print(f"Invalid character: {ch}")
        break
else:
    if len(stack) == 1:
        print("Postfix Expression:", exp)
        print("Evaluated Result   :", stack[0])
    else:
        print("Invalid postfix expression!")
```

Space for Output

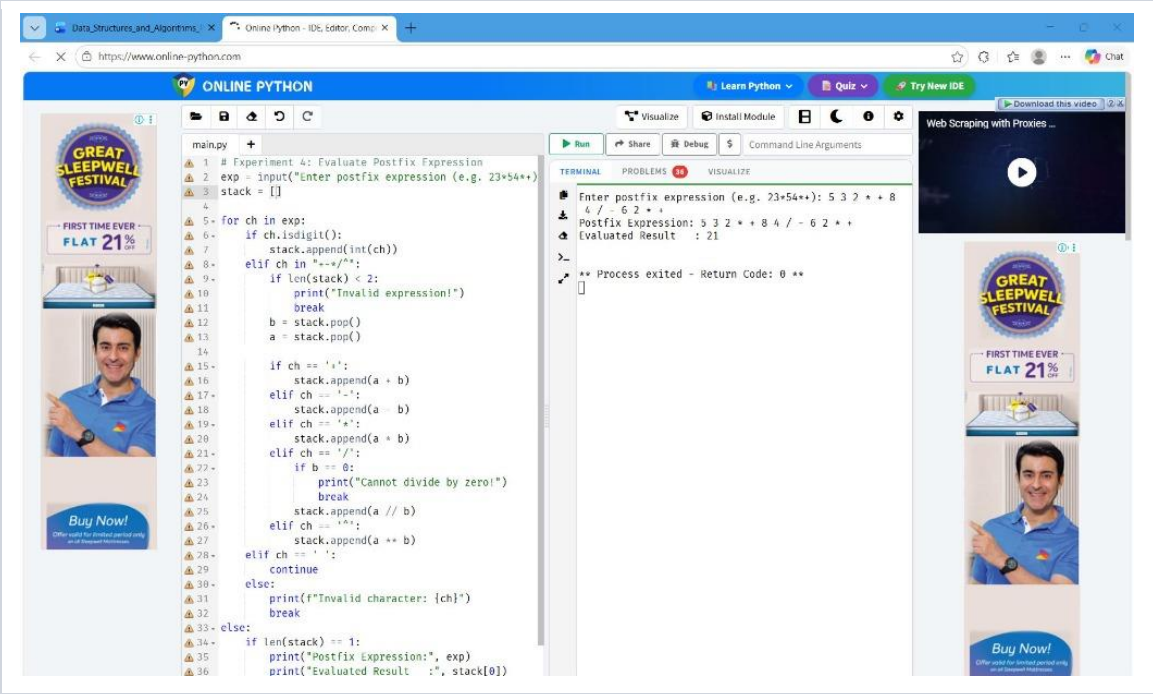


Figure: Execution output of Evaluate Postfix Expression

Sample Run: Execution Log / Console Output (Terminal Execution):

```
Enter postfix expression (e.g. 23*54*+): 5 3 2 * + 8 4 / - 6 2 * +
Postfix Expression: 5 3 2 * + 8 4 / - 6 2 * +
Evaluated Result   : 21

** Process exited - Return Code: 0 **
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 5: CONVERT INFIX EXPRESSION TO POSTFIX USING STACK

AIM: To write a Python program that converts an Infix mathematical expression to a Postfix expression using the stack-based Shunting-Yard algorithm.

Python Source Code

```
# Experiment 5: Convert Infix to Postfix
def priority(op):
    if op == '^':
        return 3
    if op in "*/":
        return 2
    if op in "+-":
        return 1
    return 0

exp = input("Enter infix expression: ")
stack = []
postfix = ""

for ch in exp:
    if ch.isalnum():
        postfix += ch
    elif ch == '(':
        stack.append(ch)
    elif ch == ')':
        while stack and stack[-1] != '(':
            postfix += stack.pop()
        if stack and stack[-1] == '(':
            stack.pop()
    elif ch in "+-*/^":
        while (stack and stack[-1] != '(' and
               priority(stack[-1]) >= priority(ch)):
            postfix += stack.pop()
        stack.append(ch)

while stack:
    postfix += stack.pop()

print("Infix expression  :", exp)
print("Postfix expression:", postfix)
```

Space for Output

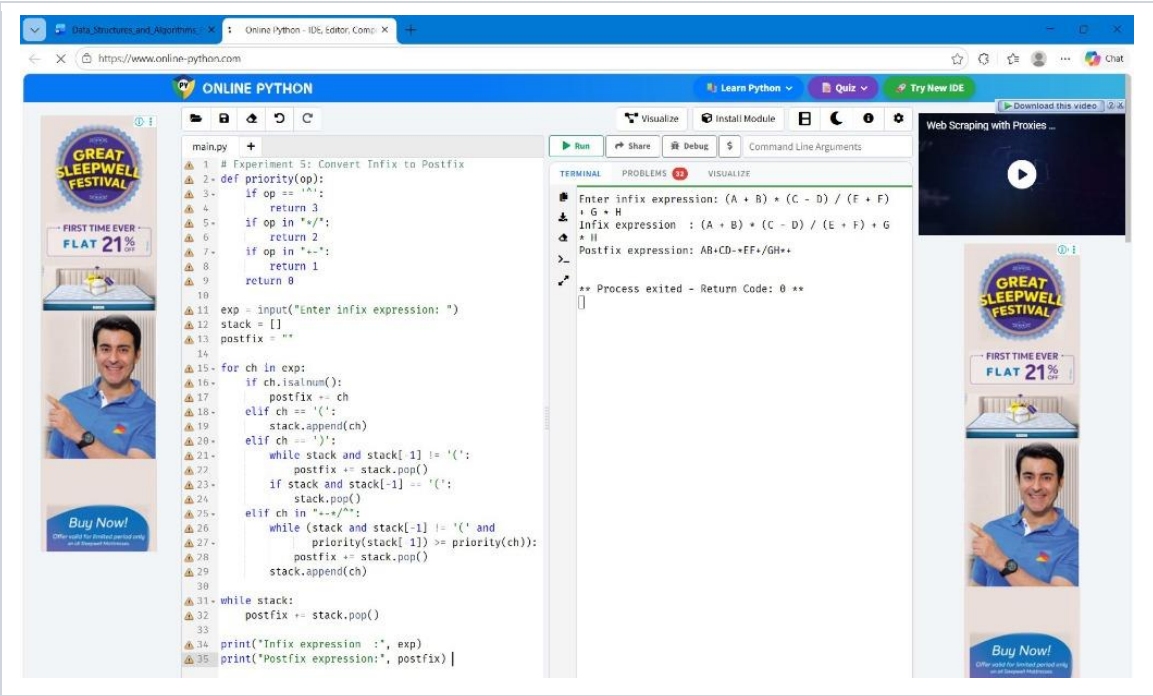


Figure: Execution output of Convert Infix to Postfix

Sample Run: Execution Log / Console Output (Terminal Execution):

```
Enter infix expression: (A + B) * (C - D) / (E + F) + G * H
Infix expression  : (A + B) * (C - D) / (E + F) + G * H
Postfix expression: AB+CD-*EF+/GH*+

** Process exited - Return Code: 0 **
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 6: SINGLY LINKED LIST OPERATIONS (INSERT, DELETE, DISPLAY)

AIM: To implement a Singly Linked List with operations: inserting a node at the end, deleting a node by value, and displaying all nodes.

Python Source Code

```
# Experiment 6: Singly Linked List (Insert, Delete, Display)
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = None

while True:
    print("\n--- SINGLY LINKED LIST MENU ---")
    print("1. Insert at end")
    print("2. Delete by value")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value to insert: "))
        new_node = Node(value)
        if head is None:
            head = new_node
        else:
            temp = head
            while temp.next is not None:
                temp = temp.next
            temp.next = new_node
        print(f"Node {value} inserted at end.")

    elif choice == 2:
        value = int(input("Enter value to delete: "))
        if head is None:
            print("List is empty!")
        elif head.data == value:
            head = head.next
            print(f"Deleted {value} from head.")
        else:
            temp = head
            while temp.next is not None and temp.next.data != value:
                temp = temp.next
            if temp.next is not None:
                temp.next = temp.next.next
                print(f"Deleted {value} from list.")
            else:
                print("Value not found in list!")

    elif choice == 3:
        if head is None:
            print("List is empty.")
        else:
```

```
            print("Linked List:", end=" ")
            temp = head
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

    elif choice == 4:
        print("Exiting...")
        break
    else:
        print("Invalid choice!")
```

Space for Output

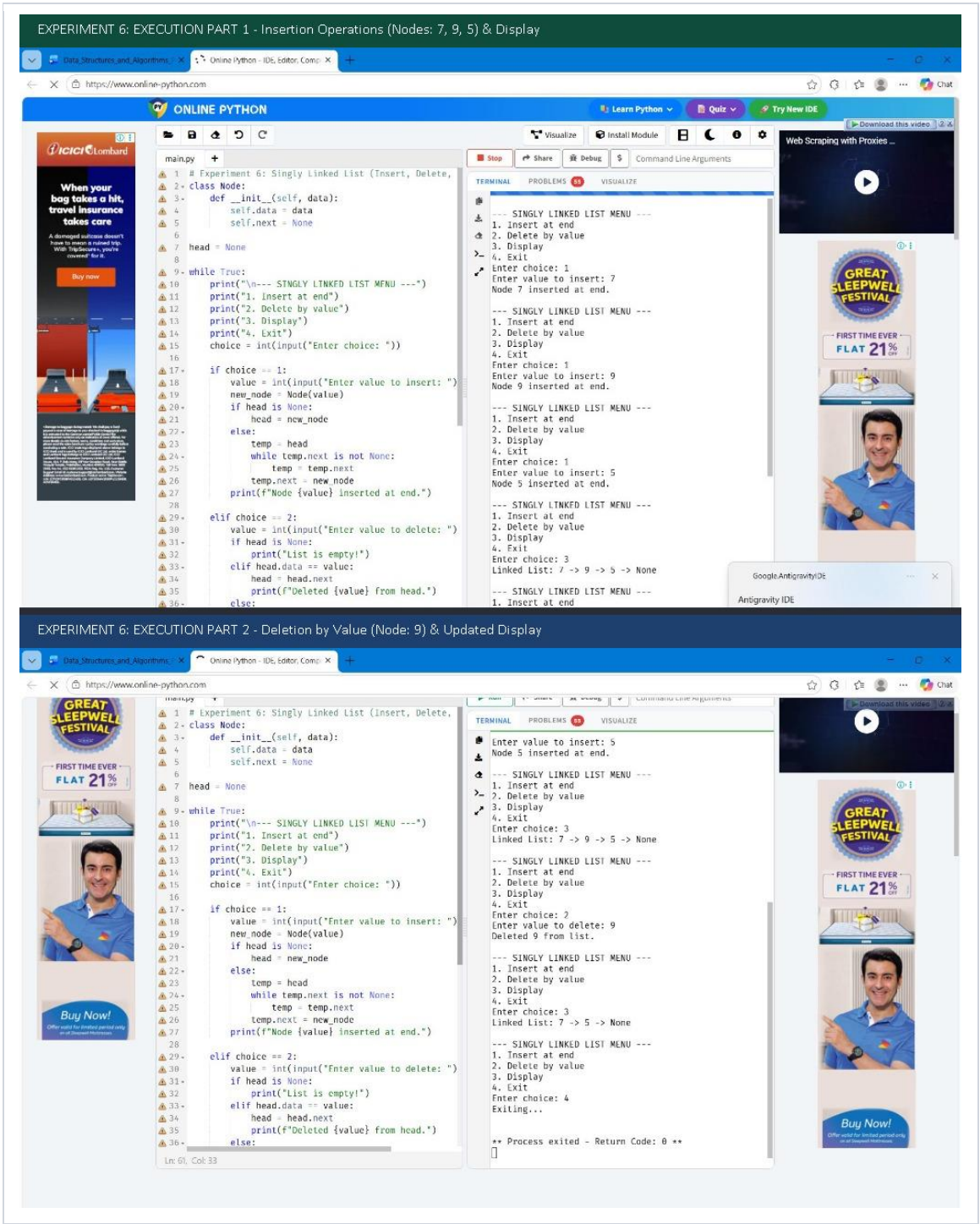


Figure: Execution output of Singly Linked List (Insert, Delete, Display)

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- SINGLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
```

```
3. Display
4. Exit
Enter choice: 1
Enter value to insert: 7
Node 7 inserted at end.

--- SINGLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 1
Enter value to insert: 9
Node 9 inserted at end.

--- SINGLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 1
Enter value to insert: 5
Node 5 inserted at end.

--- SINGLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 3
Linked List: 7 -> 9 -> 5 -> None

--- SINGLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 2
Enter value to delete: 9
Deleted 9 from list.

--- SINGLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 3
Linked List: 7 -> 5 -> None

--- SINGLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 4
Exiting...
```

```
** Process exited - Return Code: 0 **
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 7: CIRCULAR LINKED LIST OPERATIONS (INSERT, DELETE, DISPLAY)

AIM: To implement a Circular Singly Linked List with operations: inserting a node at the end, deleting a node by value, and displaying the circular structure.

Python Source Code

```
# Experiment 7: Circular Linked List (Insert, Delete, Display)
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

last = None

while True:
    print("\n--- CIRCULAR LINKED LIST MENU ---")
    print("1. Insert at end")
    print("2. Delete by value")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value to insert: "))
        new_node = Node(value)
        if last is None:
            last = new_node
            new_node.next = new_node
        else:
            new_node.next = last.next
            last.next = new_node
            last = new_node
        print(f"Node {value} inserted at end.")

    elif choice == 2:
        value = int(input("Enter value to delete: "))
        if last is None:
            print("List is empty!")
        else:
            current = last.next
            previous = last
            found = False
            while True:
                if current.data == value:
                    found = True
                    if current == last and current.next == current:
                        last = None
                    else:
                        previous.next = current.next
                        if current == last:
                            last = previous
                    print(f"Deleted {value} from circular list.")
                    break
                previous = current
                current = current.next
```

```
            if current == last.next:
                break
        if not found:
            print("Value not found in circular list!")

elif choice == 3:
    if last is None:
        print("Circular Linked List is empty.")
    else:
        print("Circular Linked List:", end=" ")
        temp = last.next
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == last.next:
                break
        print("(head)")

elif choice == 4:
    print("Exiting...")
    break
else:
    print("Invalid choice!")
```

Space for Output

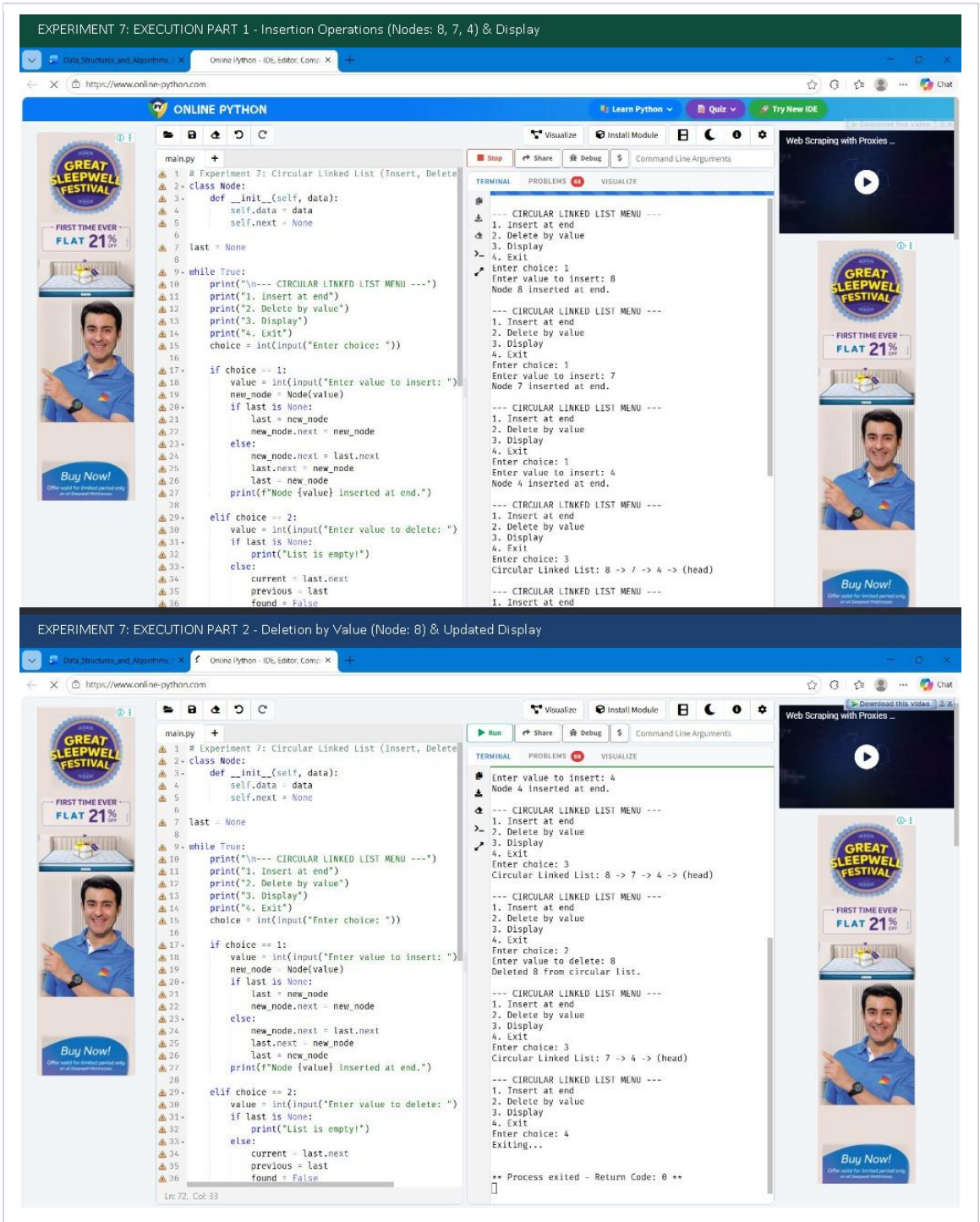


Figure: Execution output of Circular Linked List (Insert, Delete, Display)

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- CIRCULAR LINKED LIST MENU ---
1. Insert at end
2. Delete by value
```

```
3. Display
4. Exit
Enter choice: 1
Enter value to insert: 8
Node 8 inserted at end.

--- CIRCULAR LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 1
Enter value to insert: 7
Node 7 inserted at end.

--- CIRCULAR LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 1
Enter value to insert: 4
Node 4 inserted at end.

--- CIRCULAR LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 3
Circular Linked List: 8 -> 7 -> 4 -> (head)

--- CIRCULAR LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 2
Enter value to delete: 8
Deleted 8 from circular list.

--- CIRCULAR LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 3
Circular Linked List: 7 -> 4 -> (head)

--- CIRCULAR LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 4
Exiting...
```

```
** Process exited - Return Code: 0 **
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 8A: STACK IMPLEMENTATION USING LINKED LIST

AIM: To implement a dynamically allocated Stack data structure using a Singly Linked List supporting Push, Pop, and Display operations.

Python Source Code

```
# Experiment 8A: Stack using Linked List
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

top = None

while True:
    print("\n--- LINKED STACK MENU ---")
    print("1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        x = int(input("Enter value to push: "))
        new_node = Node(x)
        new_node.next = top
        top = new_node
        print(f"Pushed {x} onto stack.")

    elif choice == 2:
        if top is None:
            print("Stack Underflow (empty stack)!")
        else:
            print("Popped value:", top.data)
            top = top.next

    elif choice == 3:
        if top is None:
            print("Stack is empty.")
        else:
            print("Stack elements (top to bottom):", end=" ")
            temp = top
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

    elif choice == 4:
        print("Exiting...")
        break
    else:
        print("Invalid choice!")
```

Space for Output

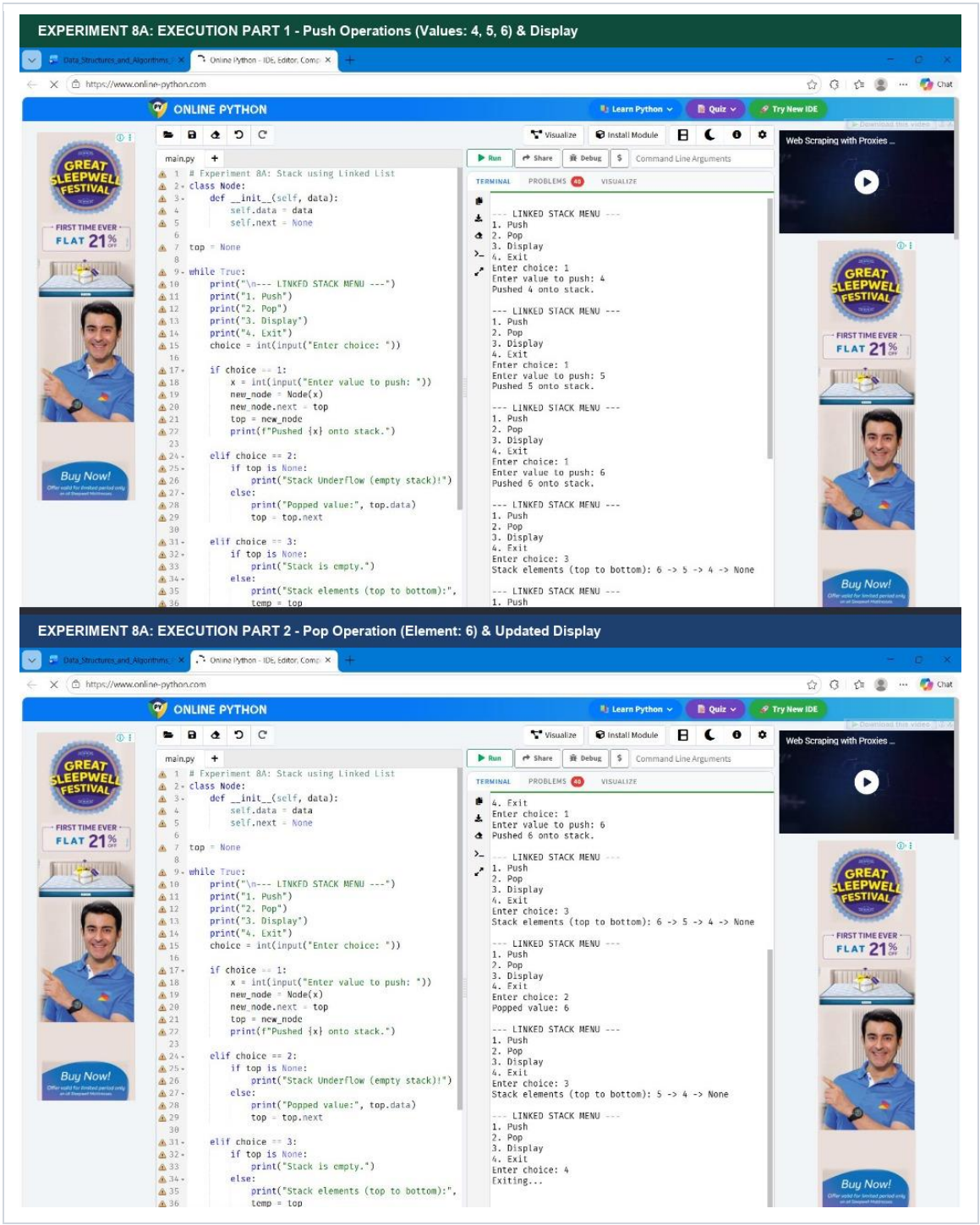


Figure: Execution output of Stack using Linked List

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- LINKED STACK MENU ---
1. Push
2. Pop
```

```
3. Display
4. Exit
Enter choice: 1
Enter value to push: 4
Pushed 4 onto stack.

--- LINKED STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter value to push: 5
Pushed 5 onto stack.

--- LINKED STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 1
Enter value to push: 6
Pushed 6 onto stack.

--- LINKED STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack elements (top to bottom): 6 -> 5 -> 4 -> None

--- LINKED STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 2
Popped value: 6

--- LINKED STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 3
Stack elements (top to bottom): 5 -> 4 -> None

--- LINKED STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter choice: 4
Exiting...
```

```
** Process exited - Return Code: 0 **
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 8B: QUEUE IMPLEMENTATION USING LINKED LIST

AIM: To implement a Queue data structure using a Singly Linked List with front and rear pointers supporting Enqueue, Dequeue, and Display.

Python Source Code

```
# Experiment 8B: Queue using Linked List
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

front = None
rear = None

while True:
    print("\n--- LINKED QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        x = int(input("Enter value to enqueue: "))
        new_node = Node(x)
        if rear is None:
            front = rear = new_node
        else:
            rear.next = new_node
            rear = new_node
        print(f"Enqueued {x} into queue.")

    elif choice == 2:
        if front is None:
            print("Queue Underflow (empty queue)!")
        else:
            print("Dequeued value:", front.data)
            front = front.next
            if front is None:
                rear = None

    elif choice == 3:
        if front is None:
            print("Queue is empty.")
        else:
            print("Queue elements (front to rear):", end=" ")
            temp = front
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

    elif choice == 4:
        print("Exiting...")
```

```
        break
    else:
        print("Invalid choice!")
```

Space for Output

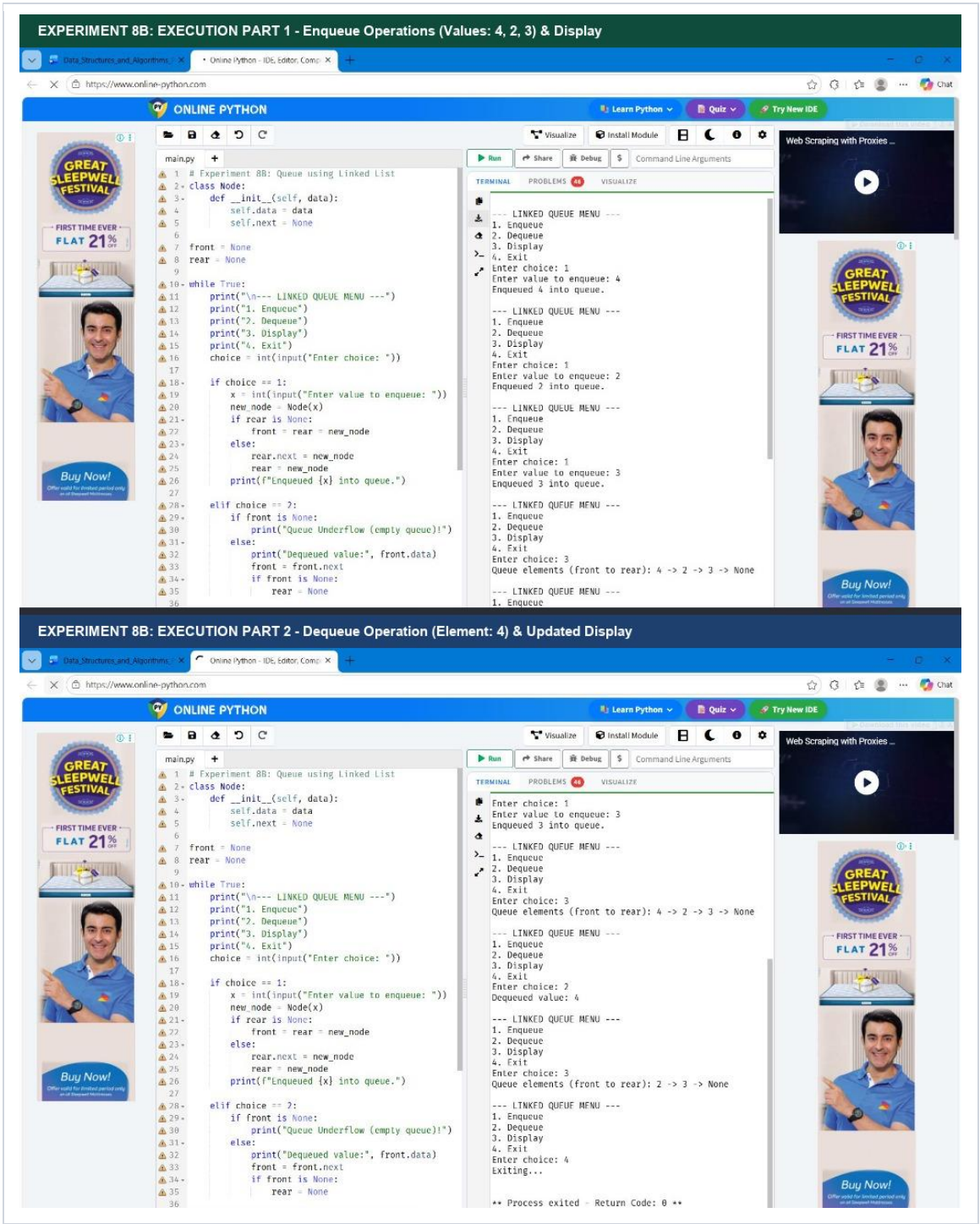


Figure: Execution output of Queue using Linked List

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- LINKED QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter value to enqueue: 4
Enqueued 4 into queue.

--- LINKED QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter value to enqueue: 2
Enqueued 2 into queue.

--- LINKED QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter value to enqueue: 3
Enqueued 3 into queue.

--- LINKED QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 3
Queue elements (front to rear): 4 -> 2 -> 3 -> None

--- LINKED QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 2
Dequeued value: 4

--- LINKED QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 3
Queue elements (front to rear): 2 -> 3 -> None

--- LINKED QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
```

```
4. Exit
Enter choice: 4
Exiting...

** Process exited - Return Code: 0 **
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 8C: CIRCULAR QUEUE IMPLEMENTATION USING LINKED LIST

AIM: To implement a Circular Queue using a Linked List maintained with a single rear pointer.

Python Source Code

```
# Experiment 8C: Circular Queue using Linked List
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

rear = None

while True:
    print("\n--- LINKED CIRCULAR QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        x = int(input("Enter value: "))
        new_node = Node(x)
        if rear is None:
            rear = new_node
            new_node.next = new_node
        else:
            new_node.next = rear.next
            rear.next = new_node
            rear = new_node
        print(f"Enqueued {x} in circular queue.")

    elif choice == 2:
        if rear is None:
            print("Circular Queue Underflow!")
        else:
            first = rear.next
            print("Dequeued value:", first.data)
            if first == rear:
                rear = None
            else:
                rear.next = first.next

    elif choice == 3:
        if rear is None:
            print("Circular Queue is empty.")
        else:
            print("Circular Queue elements:", end=" ")
            temp = rear.next
            while True:
                print(temp.data, end=" -> ")
                temp = temp.next
                if temp == rear.next:
                    break
            print("(front)")
```

```
    elif choice == 4:
        print("Exiting...")
        break
    else:
        print("Invalid choice!")
```

Space for Output

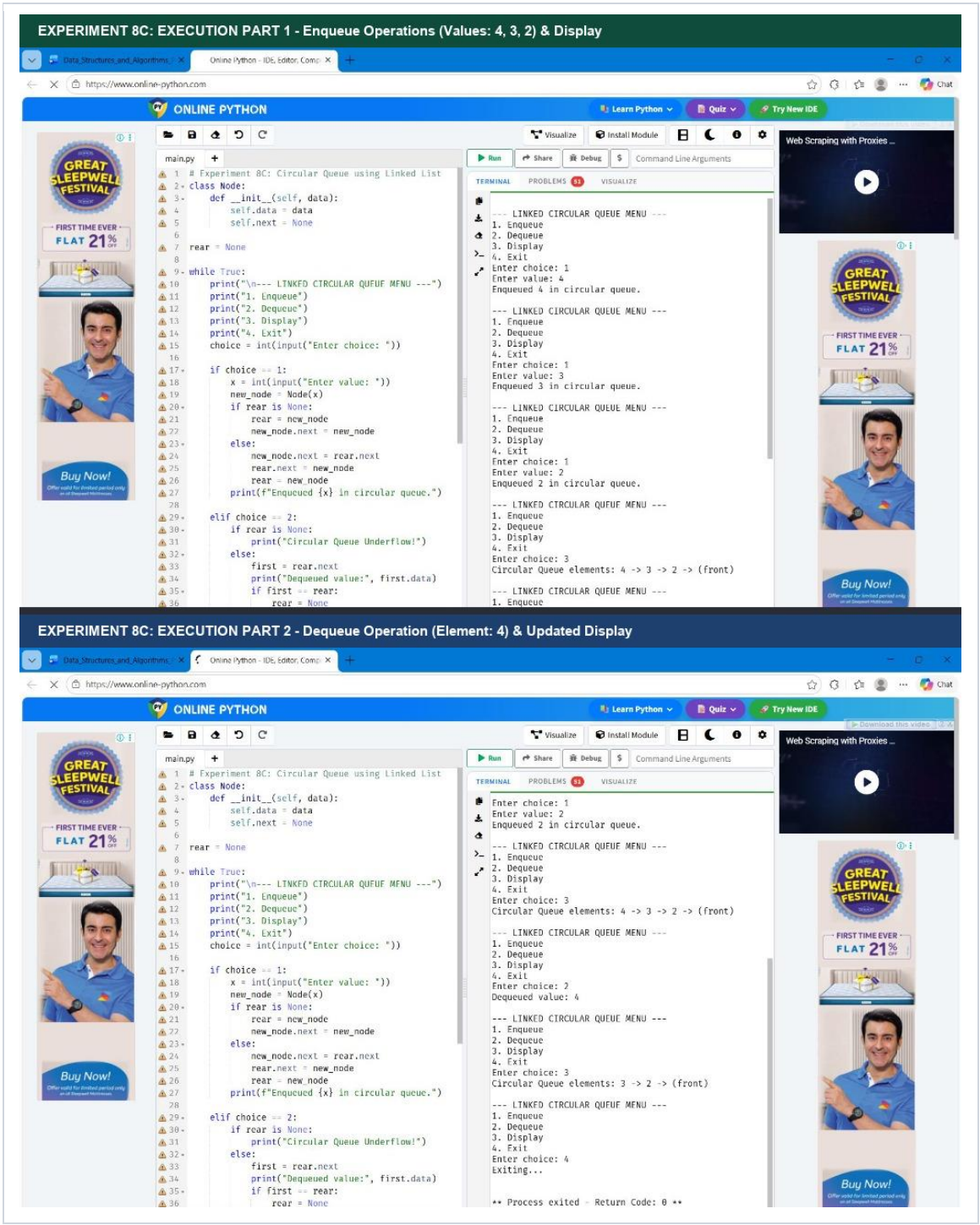


Figure: Execution output of Circular Queue using Linked List

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- LINKED CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
```

```
3. Display
4. Exit
Enter choice: 1
Enter value: 4
Enqueued 4 in circular queue.

--- LINKED CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter value: 3
Enqueued 3 in circular queue.

--- LINKED CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 1
Enter value: 2
Enqueued 2 in circular queue.

--- LINKED CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 3
Circular Queue elements: 4 -> 3 -> 2 -> (front)

--- LINKED CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 2
Dequeued value: 4

--- LINKED CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 3
Circular Queue elements: 3 -> 2 -> (front)

--- LINKED CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter choice: 4
Exiting...
```

```
** Process exited - Return Code: 0 **
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 9A: CONCATENATION OF TWO SINGLY LINKED LISTS

AIM: To write a Python program that creates two separate singly linked lists and concatenates the second list to the end of the first list.

Python Source Code

```
# Experiment 9A: Concatenation of Two Linked Lists
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def make_list(n, name="list"):
    head = None
    tail = None
    print(f"Enter {n} elements for {name}:")
    for i in range(n):
        x = int(input(f"  Element {i + 1}: "))
        new_node = Node(x)
        if head is None:
            head = tail = new_node
        else:
            tail.next = new_node
            tail = new_node
    return head

def display_list(head):
    temp = head
    while temp is not None:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")

# Driver Code
n = int(input("Enter number of elements in first list: "))
first = make_list(n, "First List")

m = int(input("Enter number of elements in second list: "))
second = make_list(m, "Second List")

print("\nFirst List :", end=" ")
display_list(first)
print("Second List:", end=" ")
display_list(second)

# Concatenation
if first is None:
    first = second
else:
    temp = first
    while temp.next is not None:
        temp = temp.next
    temp.next = second

print("\nCombined Concatenated List:", end=" ")
```

```
display_list(first)
```

Space for Output

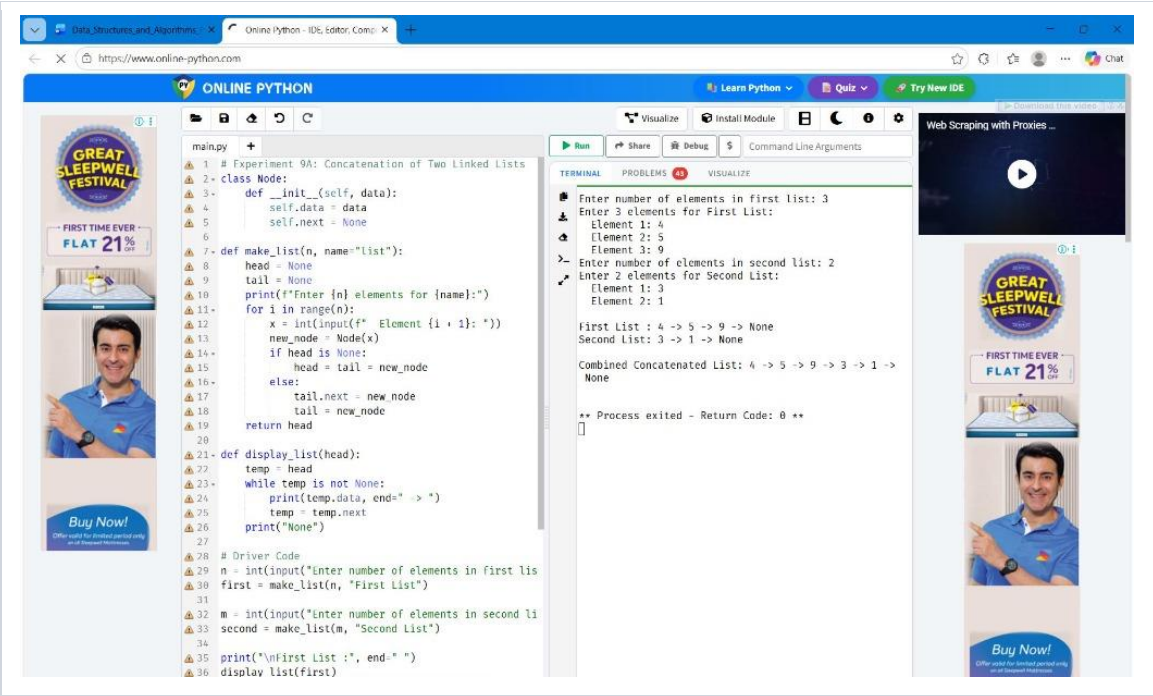


Figure: Execution output of Concatenation of Two Linked Lists

Sample Run: Execution Log / Console Output (Terminal Execution):

```
Enter number of elements in first list: 3
Enter 3 elements for First List:
  Element 1: 4
  Element 2: 5
  Element 3: 9
Enter number of elements in second list: 2
Enter 2 elements for Second List:
  Element 1: 3
  Element 2: 1

First List : 4 -> 5 -> 9 -> None
Second List: 3 -> 1 -> None

Combined Concatenated List: 4 -> 5 -> 9 -> 3 -> 1 -> None

** Process exited - Return Code: 0 **
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 9B: POLYNOMIAL ADDITION USING LINKED LISTS

AIM: To represent two single-variable polynomials using linked lists and compute their sum by combining coefficients of matching exponents.

Python Source Code

```
# Experiment 9B: Polynomial Addition using Linked Lists
class Term:
    def __init__(self, coef, power):
        self.coef = coef
        self.power = power
        self.next = None

def add_term(head, coef, power):
    if coef == 0:
        return head
    new_term = Term(coef, power)
    if head is None:
        return new_term
    temp = head
    while temp.next is not None:
        temp = temp.next
    temp.next = new_term
    return head

def read_polynomial(name):
    head = None
    n = int(input(f"Number of terms in {name}: "))
    for i in range(n):
        coef = int(input(f"  Term {i + 1} Coefficient: "))
        power = int(input(f"  Term {i + 1} Power: "))
        head = add_term(head, coef, power)
    return head

def display_poly(head):
    if head is None:
        print("0")
        return
    temp = head
    while temp is not None:
        print(f"{temp.coef}x^{temp.power}", end="")
        if temp.next is not None:
            print(" + ", end="")
        temp = temp.next
    print()

def add_polynomials(p1, p2):
    result = None
    t1 = p1
    while t1 is not None:
        t2 = p2
        found = False
        while t2 is not None:
            if t1.power == t2.power:
                result = add_term(result, t1.coef + t2.coef, t1.power)
```

```
                found = True
                break
            t2 = t2.next
        if not found:
            result = add_term(result, t1.coef, t1.power)
        t1 = t1.next

    t2 = p2
    while t2 is not None:
        t1 = p1
        found = False
        while t1 is not None:
            if t2.power == t1.power:
                found = True
                break
            t1 = t1.next
        if not found:
            result = add_term(result, t2.coef, t2.power)
        t2 = t2.next
    return result

# Driver Code
p1 = read_polynomial("Polynomial 1")
p2 = read_polynomial("Polynomial 2")

print("\nPolynomial 1:", end=" ")
display_poly(p1)
print("Polynomial 2:", end=" ")
display_poly(p2)

result = add_polynomials(p1, p2)
print("Sum of Polynomials:", end=" ")
display_poly(result)
```

Space for Output

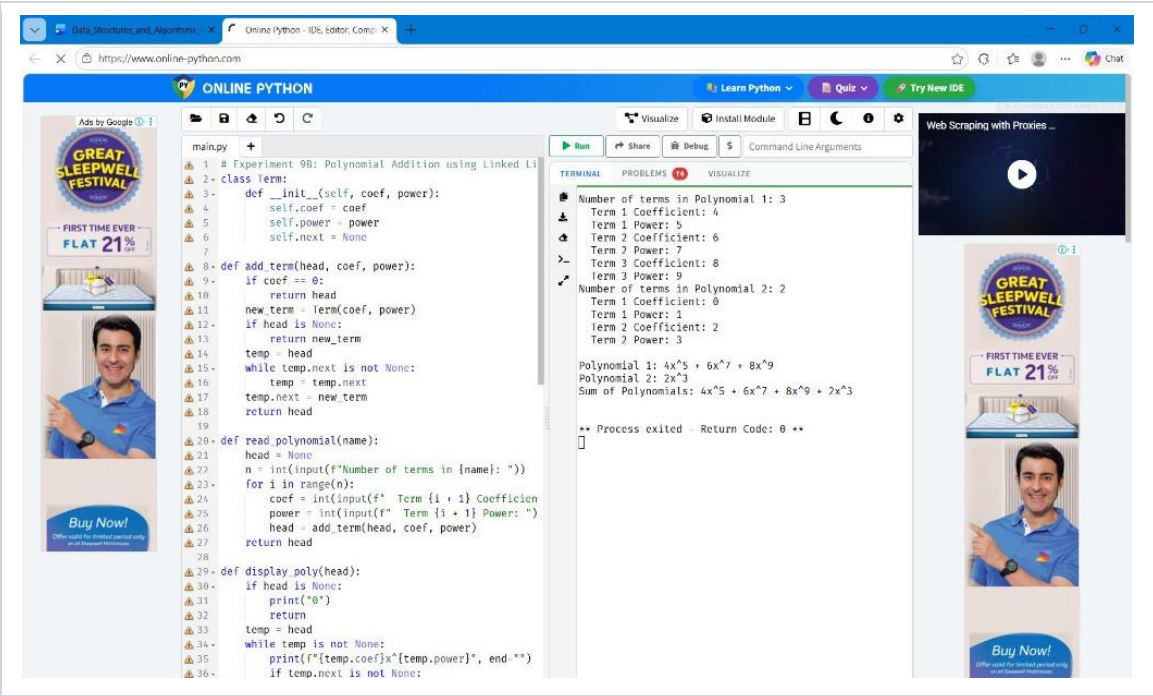


Figure: Execution output of Polynomial Addition using Linked Lists

Sample Run: Execution Log / Console Output (Terminal Execution):

```
Number of terms in Polynomial 1: 3
  Term 1 Coefficient: 4
  Term 1 Power: 5
  Term 2 Coefficient: 6
  Term 2 Power: 7
  Term 3 Coefficient: 8
  Term 3 Power: 9
Number of terms in Polynomial 2: 2
  Term 1 Coefficient: 0
  Term 1 Power: 1
  Term 2 Coefficient: 2
  Term 2 Power: 3

Polynomial 1: 4x^5 + 6x^7 + 8x^9
Polynomial 2: 2x^3
Sum of Polynomials: 4x^5 + 6x^7 + 8x^9 + 2x^3

** Process exited - Return Code: 0 **
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.

EXPERIMENT 10: DOUBLY LINKED LIST OPERATIONS (INSERT, DELETE, DISPLAY)

AIM: To implement a Doubly Linked List with operations: inserting a node at the end, deleting a node by value, and displaying nodes in forward sequence.

Python Source Code

```
# Experiment 10: Doubly Linked List (Insert, Delete, Display)
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

head = None

while True:
    print("\n--- DOUBLY LINKED LIST MENU ---")
    print("1. Insert at end")
    print("2. Delete by value")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        x = int(input("Enter value to insert: "))
        new_node = Node(x)
        if head is None:
            head = new_node
        else:
            temp = head
            while temp.next is not None:
                temp = temp.next
            temp.next = new_node
            new_node.prev = temp
        print(f"Node {x} inserted at end.")

    elif choice == 2:
        x = int(input("Enter value to delete: "))
        temp = head
        while temp is not None and temp.data != x:
            temp = temp.next

        if temp is None:
            print("Value not found in list!")
        else:
            if temp.prev is None:
                head = temp.next
                if head is not None:
                    head.prev = None
            else:
                temp.prev.next = temp.next
                if temp.next is not None:
                    temp.next.prev = temp.prev
            print(f"Deleted {x} from doubly linked list.")
```

```
    elif choice == 3:
        if head is None:
            print("List is empty.")
        else:
            print("Doubly Linked List:", end=" ")
            temp = head
            while temp is not None:
                print(temp.data, end=" <-> ")
                temp = temp.next
            print("None")

    elif choice == 4:
        print("Exiting...")
        break
    else:
        print("Invalid choice!")
```

Space for Output

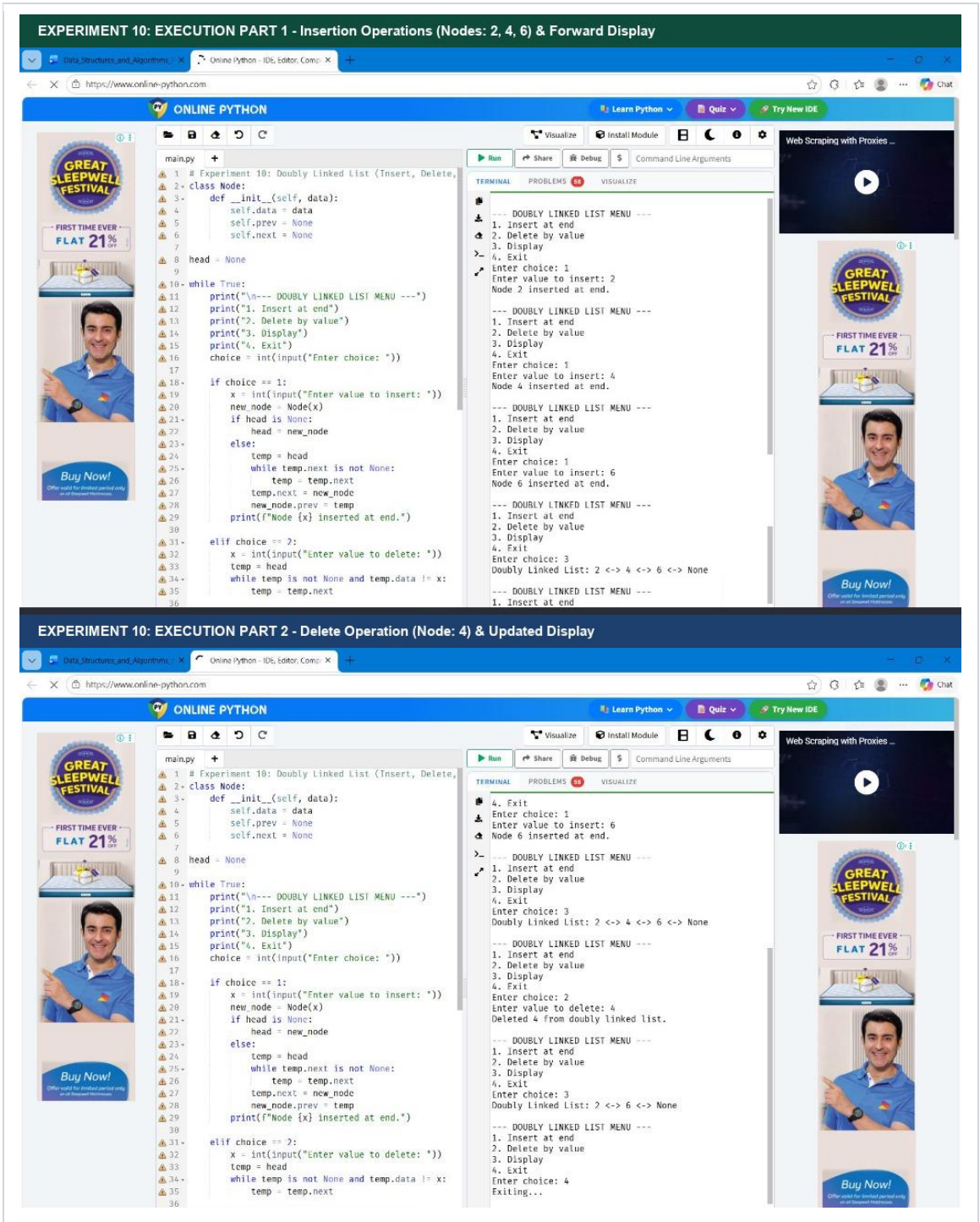


Figure: Execution output of Doubly Linked List (Insert, Delete, Display)

Sample Run: Execution Log / Console Output (Terminal Execution):

```
--- DOUBLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
```

```
3. Display
4. Exit
Enter choice: 1
Enter value to insert: 2
Node 2 inserted at end.

--- DOUBLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 1
Enter value to insert: 4
Node 4 inserted at end.

--- DOUBLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 1
Enter value to insert: 6
Node 6 inserted at end.

--- DOUBLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 3
Doubly Linked List: 2 <-> 4 <-> 6 <-> None

--- DOUBLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 2
Enter value to delete: 4
Deleted 4 from doubly linked list.

--- DOUBLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 3
Doubly Linked List: 2 <-> 6 <-> None

--- DOUBLY LINKED LIST MENU ---
1. Insert at end
2. Delete by value
3. Display
4. Exit
Enter choice: 4
Exiting...
```

Result: The program was implemented and executed successfully. All operations and output results were verified against expected algorithmic behavior.